



UNIVERSITY
OF HULL

Live Software Updates in a Real-Time System

Nathan Rignall

This dissertation is submitted on April, 2024

Word count: 3744

Abstract

My abstract ...

Acknowledgements

My acknowledgements ...

Contents

1	Introduction	8
1.1	Project Background	8
1.2	Project Objectives	8
1.3	Technical Background	9
1.3.1	Real-Time	9
1.3.2	Control Systems	9
1.3.3	Operating Systems	9
1.3.4	Mixed Criticality	10
1.3.5	Middleware	10
1.3.6	Software Updates	10
2	Literature Review	11
2.1	Review Strategy	11
2.2	Case Studies	12
2.2.1	Live & Seamless Firmware Upgrade in Real Time Control Systems .	12
2.2.2	Dynamic Software Updates for Real-Time Systems	14
2.2.3	Disruption-free Software Updates in Automation Systems	16
2.2.4	A Model for Updating Real-Time Applications	17
2.2.5	Enhanced Uptime and Firmware Cybersecurity for Grid-Connected Power Electronics	18
2.3	Project Rational	19
3	Requirements	20
3.1	Software Infrastructure	21
3.1.1	Functional Requirements	21
3.1.1.1	High-Level	21
3.1.1.2	Low-Level	22
3.1.2	Non-Functional Requirements	24
3.2	Test Harness	25

3.2.0.1	High-Level	25
3.3	Demonstration Components	26
3.3.0.1	High-Level	26
4	Design	27
4.1	Runtime	28
4.2	Scheduler Signalling	30
4.2.1	Shared Memory	31
4.2.2	Sockets	32
4.2.3	Signalling	33
4.2.4	Evaluation	34
4.3	Hardware	35
4.3.1	Test Harness	36
4.3.2	Development Cards	39
5	Implementation	41
6	Testing	43
7	Evaluation	44
8	Conclusion	45
	References	47
A	Extra Information	49

Chapter 1

Introduction

1.1 Project Background

1.2 Project Objectives

This project seeks to develop multiple methods and system architectures for live software updates in a real-time system via an example application. Using a timing analysis solution, such as a logic analyser, this will be tested to ensure that the real-time application meets all imposed deadlines (defined latency and jitter). The goal of this is to prove that it is feasibly possible to update software on a live system whilst maintaining real-time guarantees. Although this is a complex task the intention is to achieve this before February.

Another primary objective is to implement a reliable external testing method for a real-time environment, using existing research. This will allow for accurate measurement of the real-time behaviour of the program at all stages of the update, to validate that the upgrade is successful. Minimal external hardware is required, and the plan is to achieve this before February.

1.3 Technical Background

1.3.1 Real-Time

A real-time system is defined as a system which requires correct logical operation with completion in a configured deadline. This is in contrast to a non-real-time system which correct operation is desired but lacks configured deadlines. (Kavi, Akl, and Hurson 2009)

These real-time systems can be categorised into hard and soft. Hard real-time systems contain strict timing constraints, where a response must be calculated in a specified time-frame and a missed deadline could result in a catastrophic outcome. Soft real-time systems also have timing constraints, but a missed deadline should not result in a catastrophic outcome.

Typical hard real-time systems include aircraft or robotic control computers and soft real-time systems include media players or gaming computers. Soft real-time systems can be used in control applications, but the soft timing must be considered in the system design.

1.3.2 Control Systems

A control system Differential equation

1.3.3 Operating Systems

An Operating System (OS) is a piece of software that controls the execution of application programs on a system. Usually this involves managing resource allocations, such as a processor execution time, memory usage and generic inputs/outputs. Stallings 2018

In order to develop a real-time applications with an OS, a RTOS (Real-Time Operating System) is usually required. This is to ensure the time deadlines of an application are enforced.

Hard real time operating system are designed to run applications with absolute timing Worst-Case Execution Time Assigned a relative deadline, all tasks must complete in their deadline. Modelled mathematically to ensure that all tasks run to completion

Define the maximum delay after a deadline

Scheduling Memory

UNIX, POSIX

1.3.4 Mixed Criticality

1.3.5 Middleware

1.3.6 Software Updates

Chapter 2

Literature Review

Regular firmware upgrades requires systems to be taken offline for firmware change and restart, time consuming and breaks continuity. High availability systems typically using hardware redundancy to provide operational continuity through updates. At the cost of extra hardware. (Rao et al. 2023)

Physical tasks performed by the software are often expensive to interrupt. High down time and start-up costs. For example power substation - power outage for customers. Software is hardly updated during an embedded systems lifetime (typically of over 30 years). There are several solutions to updating software dynamically; few of these have actually been adapted to real-time systems. (Wahler, Richter, and Oriol 2009)

2.1 Review Strategy

Analysis of existing dynamic software updating techniques for safe and secure industrial control systems

Challenges in Dynamic Software Updating

A survey of dynamic software updating

Categories Hardware Handover Synchronisation State Continuity State Transformation

2.2 Case Studies

2.2.1 Live & Seamless Firmware Upgrade in Real Time Control Systems

(Rao et al. 2023) implements live firmware upgrades on a single microcontroller, without using any additional hardware or operating system. This paper describes the techniques used in both hardware and software to achieve such updates without reset (seamless).

Implementation

Key innovation, Function and vector pointer swapping

Hardware support for vector and function pointer swapping Compiler support for live updates and optimal switchover point

Persistence of state Keeping static and global variable addresses consistent between firmware versions

Updates involve Branching to the new firmware entrypoint Execute the live update initialisation routine Arrives in new main and performs initialisation This is when interrupts are temporally disabled

Hardware Multi-Bank Non Volatile Memory Interrupt Vector table swap Ram block swap Hardware LFU flag

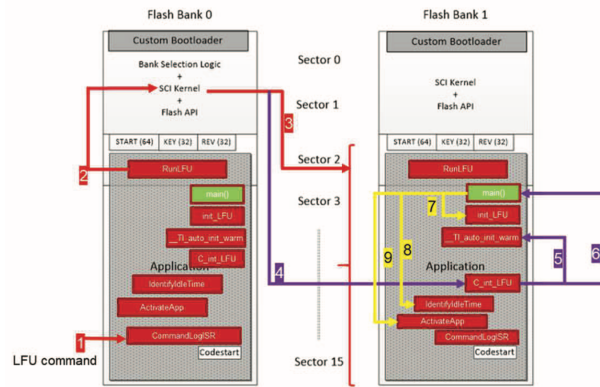


Figure 2.1: Application live firmware upgrade flow (Rao et al. 2023)

Test Case

To validate the updates they, implemented a DC-DC buck converter with a tightly regulated output voltage, typically used in server power supplies. These converters are often controlled using a digital power controller implemented on an microcontroller. A

live update must complete within the idle time between interrupts on the PSU, with less than 1us of switch time.

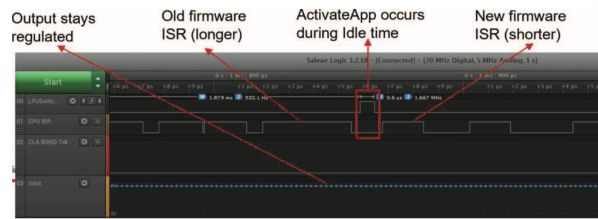


Figure 2.2: Live firmware upgrade switchover steps (Rao et al. 2023)

Evaluation

Some text

2.2.2 Dynamic Software Updates for Real-Time Systems

(Wahler, Richter, and Oriol 2009) defines a solution for updating component-based cyclic embedded systems, without violating real-time constraints.

Implementation

This implementation focused on using COTS operating systems.

They used a custom component-based framework infrastructure on top of the operating system to execute software components, effectively an embedded middleware. Each component had its own address space with script separation and defined inputs and outputs. These components communicated using message passing (channel buffers). A manager program (which was also an upgradable component) created and maintained the channels.

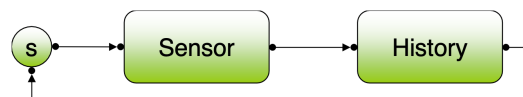


Figure 2.3: Updating component 'History' (Wahler, Richter, and Oriol 2009)

How to identify points in time when an update can occur System performs the checks at the end of every execution cycle

New update is sent to the target machine using OS functionality in a low priority thread
Upload is detected by the component manager Tells the old component Hands the connections to channels back to the component manager and terminates
Component prime acquires the connects to the channels and begins operations

Steps 1 and 2 are not required to meet and deadlines, steps 3 to 5 must execute before the end of the cycle/

How to transfer the state of a component to a new version State must be transferred in the same cycle of the handover Data types may need to be converted (celsius to kelvin)

Component manager creates an area of shared memory that is passed to components. Before a component terminates it writes its state to the area and adds version information. When a component starts it reads the memory, asses if a state transformation must occur (these are stored within the components)

Test Case

Some text

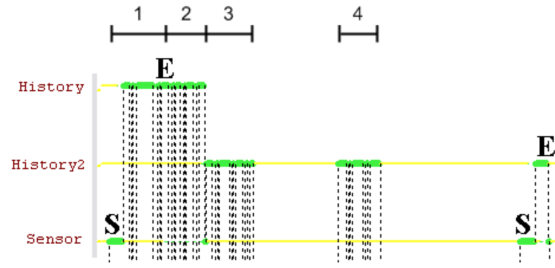


Figure 2.4: Updating component 'History' (Wahler, Richter, and Oriol 2009)

Evaluation

Successful, maximum cycle frequency of about 300hz.

2.2.3 Disruption-free Software Updates in Automation Systems

Another approach to updating software at runtime without disruption was developed by (Wahler and Oriol 2014). This uses a component based architecture developed by ABB Corporate Research (FASA), shown in Figure 2.5.

Implementation

FASA defines function blocks that can be aggregated together to form a component, only compute components are stateful. These function blocks are executed cyclically at a desired frequency and connected by communication channels. A background thread with low priority performs the management actions. Multiple FASA systems are synchronised over the network using precision time protocol (PTP.)

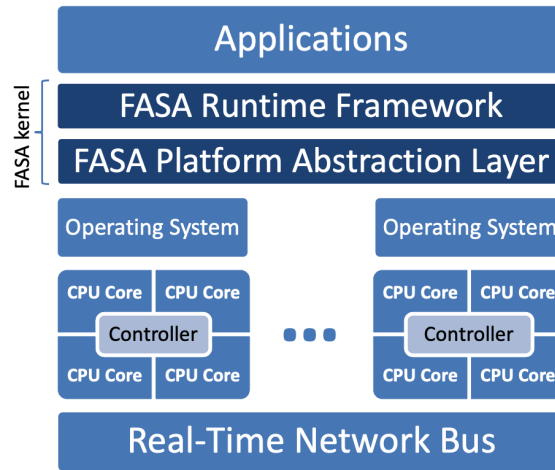


Figure 2.5: FASA system architecture (Wahler and Oriol 2014)

First the component binaries are loaded using shared libraries (DLLs), it is assumed the components are already present in the system. Next, the system is configured using configurations that define the layout of the components and channels. State is transferred between components using Teach and Learn blocks, optional type conversions occurs in these Learn blocks. Once all the controllers in the FASA network are ready to transfer to the new configuration, the switch command is issued.

Test Case

Validated the solution with a test case, magnetic levitation experiment, instable physical process Control algorithm of a magnetic levitation device at 1kHz

Evaluation

Most through implementation Cyclic application execution State transfer mechanisms

2.2.4 A Model for Updating Real-Time Applications

(Montgomery 2004)

Implementation

Text

Test Case

Text

Evaluation

Text

2.2.5 Enhanced Uptime and Firmware Cybersecurity for Grid-Connected Power Electronics

(Moquin et al. 2019)

Implementation

Text

Test Case

Text

Evaluation

Text

2.3 Project Rational

Chapter 3

Requirements

3.1 Software Infrastructure

To facilitate real-time software updates this project will develop an embedded middleware framework that shall be known as the software infrastructure. This framework's should run software components which can be dynamically updated. The high-level requirements are first defined, followed by the more in-depth low-level requirement that shall be used to implement test-cases.

3.1.1 Functional Requirements

3.1.1.1 High-Level

REQ1 The software infrastructure must provide the functionality to run software components with a cyclic schedule.

REQ1.1 The software infrastructure must allow users to create/start/stop/remove software components during operation.

REQ1.2 The software infrastructure must allow users to change the configured cyclic schedule during operation.

REQ2 The software infrastructure must provide the functionality to allow components to communicate using messages passing.

REQ2.1 The software infrastructure must allow users to configure communications between components using routes and channels.

REQ2.2 The software infrastructure must allow users to add/remove routes between software components during operation.

REQ3 The software infrastructure must provide the functionality to save and restore state between components.

REQ3.1 The software infrastructure must allow users to define which software components must save state during operation.

REQ3.2 The software infrastructure must allow users to configure which component restore from state during operation.

REQ3.3 The software infrastructure must allow users define state transformation maps so different versions of state can be restored.

3.1.1.2 Low-Level

REQ4 A software component must be defined as a software template that developers can implement, for example a Rust trait.

REQ4.1 There must be sufficient separation between components, such that if one fails, other components are unaffected, for example containers.

REQ5 The software infrastructure must execute components using a defined soft real-time schedule.

REQ5.1 The schedule must execute cyclically, with a defined deadline.

REQ5.2 The schedule must execute alternating major frames.

REQ5.3 The schedule must execute all minor frames within a major frame.

REQ5.4 A major frame must execute a single component until a defined deadline.
If the component exceeds its deadline, its operation must be suspended.

REQ6 The software infrastructure must route messages between component using channels.

REQ6.1 The message router must be configured using a routing table.

REQ6.2 The routing table must contain source and destination channels.

REQ6.3 The message router must pass messages between components when executed.

REQ7 The software infrastructure must be configured using task lists.

REQ7.1 A task list must contain a series of actions with relevant data and is either non-blocking or blocking.

REQ7.2 A blocking action must complete in one execution cycle.

REQ7.3 All blocking actions in a task must complete in the same execution cycle.

REQ7.4 A non-blocking action can take multiple execution cycles to complete.

REQ8 The software infrastructure must implement the create component non-blocking action.

REQ8.1 The create component action must first load the software component binary from storage.

- REQ8.2 The create component action must then start the software component binary as a separate process
- REQ8.3 The create component action must take the path of the software component binary and whether state sync should be enabled as parameters.
- REQ9 The software infrastructure must implement the start component blocking action.
- REQ9.1 The start component action must mark the software component as ready to run.
- REQ10 The software infrastructure must implement the stop component blocking action.
- REQ10.1 The start component action must mark the software component as not ready to run.
- REQ11 The software infrastructure must implement the add route blocking action.
- REQ11.1 The add route action must add the relevant source and destination to the routing table.
- REQ11.2 The add route action must take source component plus channel and target component plus channel as parameters.
- REQ12 The software infrastructure must implement the remove route blocking action.
- REQ12.1 The remove route action must remove the relevant source and destination from the routing table.
- REQ13 The software infrastructure must implement the set schedule blocking action.
- REQ13.1 The set schedule action must first check if the schedule is valid.
- REQ13.2 Only components that are created can be added to the schedule.
- REQ13.3 The schedule must define a execution frequency.
- REQ13.4 The schedule must define which order components execute.
- REQ14 The software infrastructure must implement the add state synchronise blocking action.
- REQ14.1 stuff
- REQ15 The software infrastructure must implement the remove state synchronise blocking action.
- REQ15.1 stuff

REQ16 The software infrastructure must implement the wait state synchronise non-blocking action.

REQ16.1 stuff

3.1.2 Non-Functional Requirements

3.2 Test Harness

To accurately test the real-time behaviour of the system, an external test harness is required. While latency and jitter can be measured on-target, with instrumentation, this does not independently measure the system's performance. The following requirements define the high-level requirements required for a successful test harness.

3.2.0.1 High-Level

REQ17 The test harness must measure the timed latency and jitter between sending/receiving data to/from the software infrastructure.

REQ17.1 The test harness must use independent software and hardware from the core infrastructure to verify the results.

REQ17.2 The test harness must use a hard real-time platform to measure timings, for example a microprocessor.

REQ17.3 The test harness must be paired with an analysis tool to visualize and analyze the measured data.

REQ18 An example software component, that is executed on the software infrastructure must interface with the test harness.

3.3 Demonstration Components

While the test harness is designed to evaluate the real-time behaviour of the system, additional software components are required to evaluate the correct behaviour of the system during update. These components must run on the software infrastructure.

3.3.0.1 High-Level

REQ19 The demonstration application

REQ20 REQ20.1 The test harness must use independent software and hardware from the core infrastructure to verify the results.

REQ20.2 The test harness must use a hard real-time platform to measure timings, for example a microprocessor.

REQ20.3 The test harness must be paired with an analysis tool to visualize and analyze the measured data.

REQ21 An example software component, that is executed on the software infrastructure must interface with the test harness.

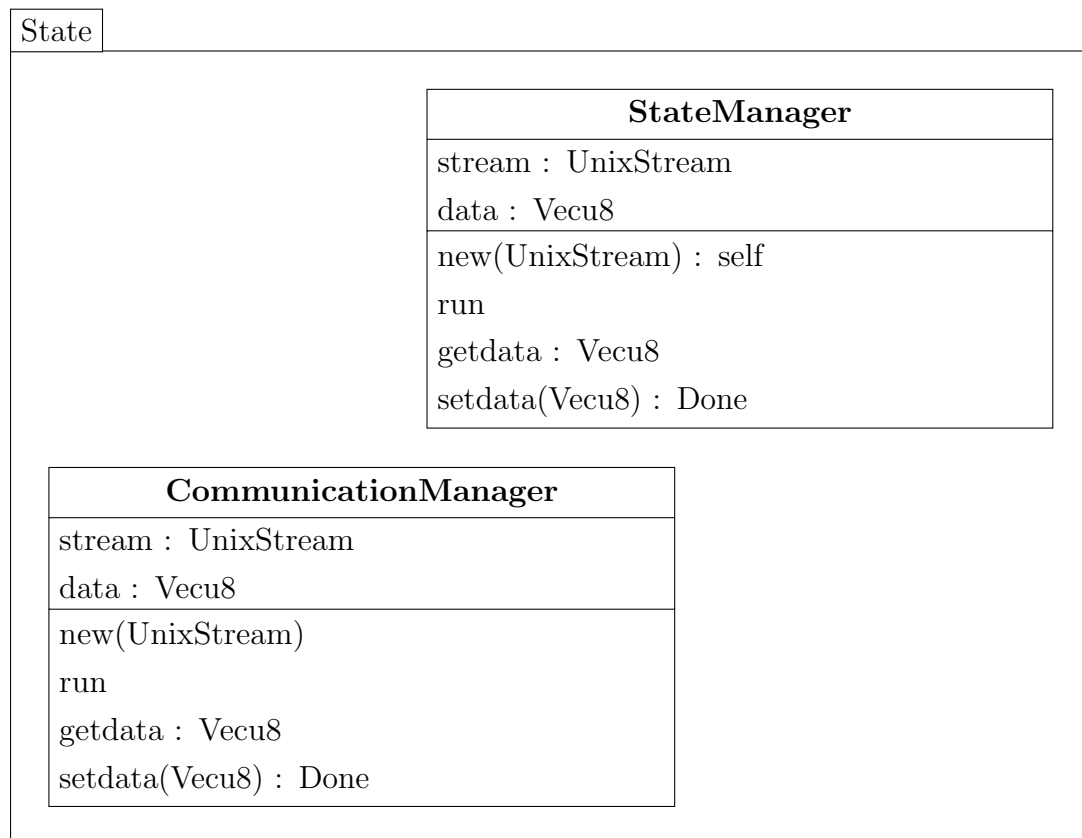
Example application that uses one differential equation and requires its state to be maintained with a plant for the example application to test it's functionality

The example application must maintained

The plant must model a mass falling due to gravity

Chapter 4

Design



4.1 Runtime

To select the most optimal environment to develop the software infrastructure upon, a series of benchmarking activities were carried out. These focused on evaluating operating systems and hypervisors which could operate in a real time environment. The search was restricted to free and open source software which supports the Raspberry Pi Compute Module 4.

EXPLAIN THE VIRTUALISATION EXTENSIONS AND VIRTUALISATION

(Abeni and Faggioli 2020) compiled a similar study into the real time performance of open source hypervisors. Comparing KVM and Xen on various intel x86 processors.

SUMMARISE THE WORK HERE

For the purpose of this project the real-time performance on an Arm64 target, the chosen development target, must be specifically analysed. It is reasonable to expect that the performance characteristics may be different between the processor architectures due to the implementation of virtualisation extensions. This analysis shall compare the behaviour of 'bare metal' Linux and the Xen hypervisor, not including KVM.

This is because KVM is implemented on top of the Linux kernel.

To reap the benefits of dynamic updates it is preferred that the hypervisor is independent of the operating system (explain why).

Also, in a true real-time system it is unusual to use linux. Rather a more preferable OS such as Zephyr or FreeRTOS (Free and open source solutions used).

Comparing straight linux and linux + Xen should give us a good method to estimate the specific latencies brought about by Xen.

EXPLAIN RT LINUX

Explain Yocto and Bitbake here too?

The RT-Tests suite includes a cyclicttest program. This is a simple yet effective program that can be used to estimate the real time latencies in a system. It measures real time jitter by creating an interrupt event at a specific time in the future and measuring the time delay between the requested wake-up and actual wake-up time. While this provides a good method for identifying operating system induced latency and jitter it does not isolate

latencies induced by hardware or low-level firmware. At this stage it can be assumed that the selected development targets do not induce significant latencies.

Under load rt-characteristics change

To set a baseline we shall first compare the real time behaviour of Linux with the standard Preemnt and Preempt RT kernels.

There is little difference between the two kernels when the system is mostly idle. However when the system is under load the differences between the kernel's schedulers becomes apparent. Especially so for memory stress versus cpu stress.

Now we are satisfied that linux, with the real time kernel patch performs adequately we can begin configuring Xen.

Exceptionally poor performance at first. Assume that is due to the nature of the Xen scheduler implementation.

EXPLAIN THE XEN SCHEDULER OPTIONS

When using the None scheduler these latencies and jitter are significantly improved

Multi core interference

The first experiments were performed on the linux kernel running without a hypervisor.

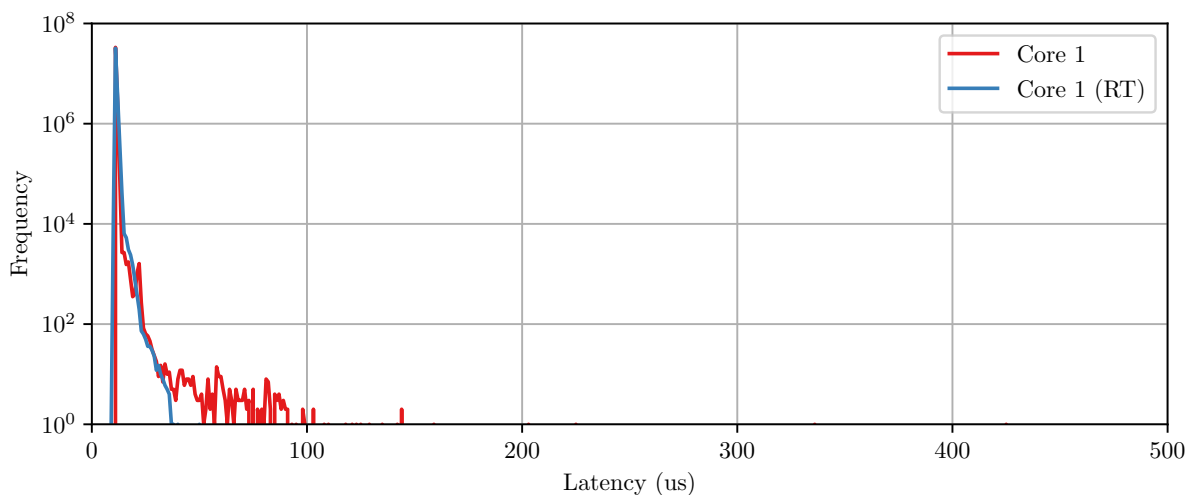


Figure 4.1: Latency Graph

4.2 Scheduler Signalling

To implement an event based scheduler, the scheduler must be able to send signals between processes without significant jitter and latency. This is because the signals can effectively send execution requests for each component. The level of jitter that is acceptable can be estimated by calculating a worst case execution time for a control loop in a particular application.

There are several different methods that can be used to send signals between processes in a Linux system. Shared memory, sockets and signalling shall all be investigated as they can be easily transported onto a non-linux POSIX system if required. This evaluation will focus on gathering latency and jitter metrics for the different techniques.

Test Program Design

Each evaluation will use a parent-child arrangement of processes, whereby a parent process spawns a child processes. These processes will both be given the highest real-time scheduler priority and pinned to two separate cores that are isolated from the operating system. The parent process shall run at a controlled frequency of 100Hz for 10,000 iterations, each iteration will request the child process to execute. At each execution, the time requested t_0 and actual execution time t_1 shall be recorded, shown in Figure 4.2. These shall be later evaluated where Δt is calculated.

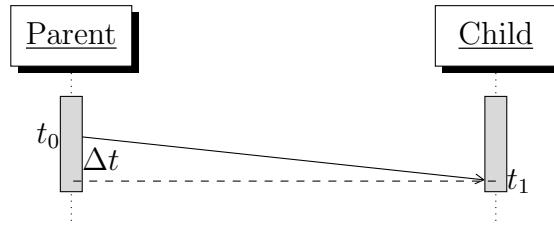


Figure 4.2: Parent-child process execution delay

Implementation

All tests were performed on the target development hardware (CM4) with Linux kernel 6.1.65 and the real-time batch applied.

4.2.1 Shared Memory

To access the shared memory functionality of Linux in the Rust language this test used the `shared_memory` crate (elast0ny 2024).

The parent process created the shared memory and a file link which the child process used to

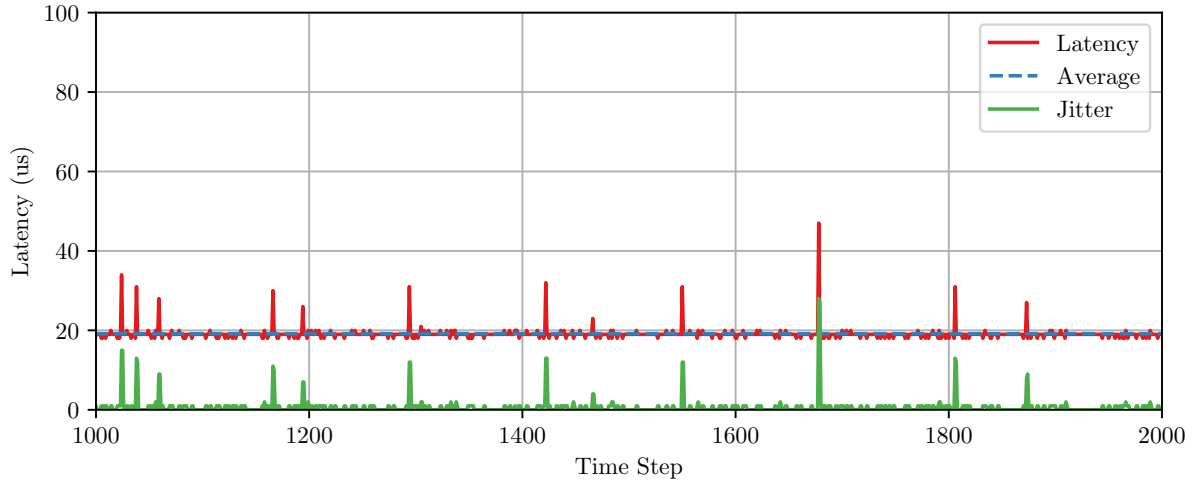


Figure 4.3: Experiment latency graph for shared memory

4.2.2 Sockets

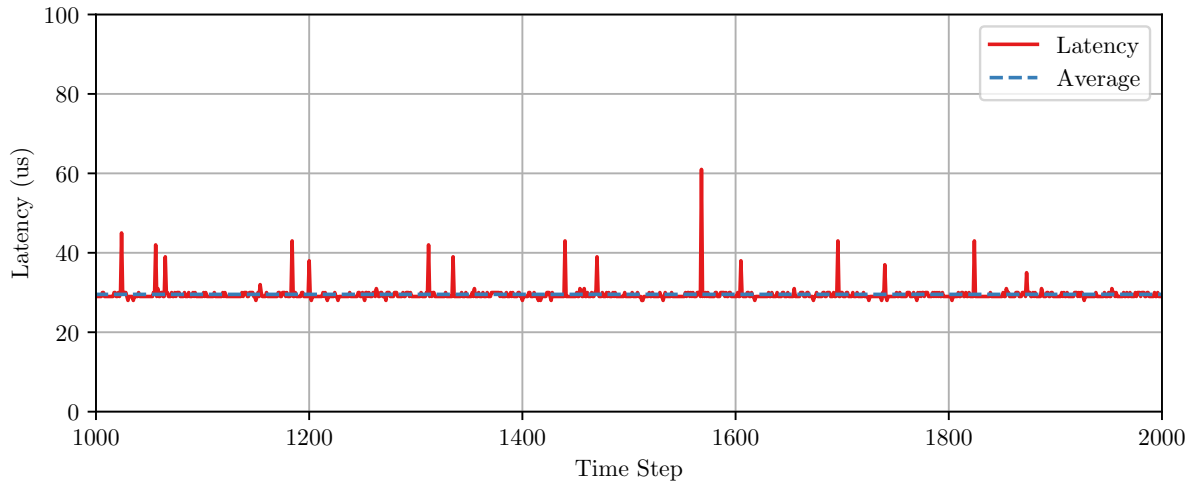


Figure 4.4: Experiment latency graph for blocking sockets

Although the blocking socket test did perform reasonably well, it was on average over 50% slower than the shared memory implementation.

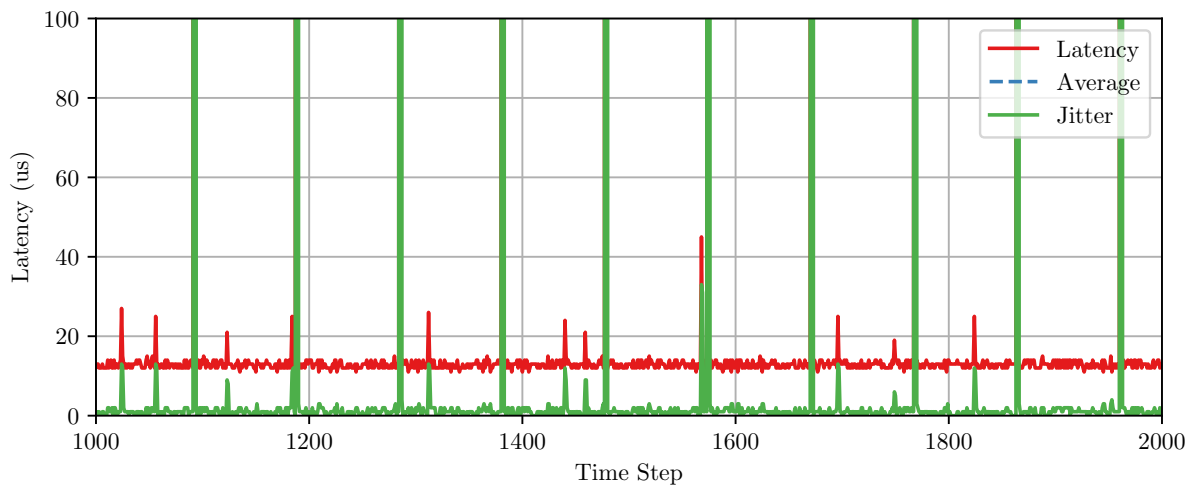


Figure 4.5: Experiment latency graph for non-blocking sockets

Something about how I planned to actually implement this.

4.2.3 Signalling

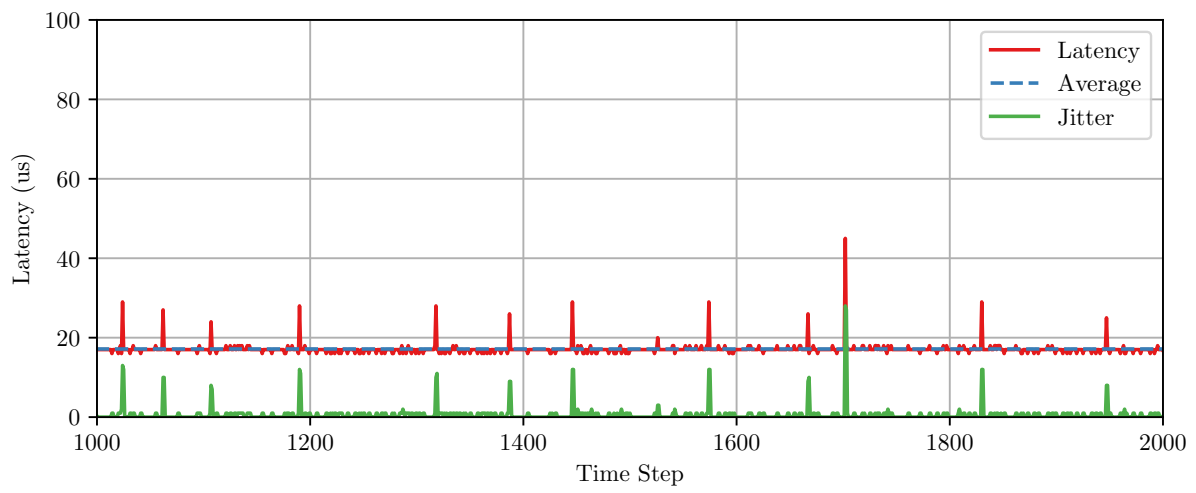


Figure 4.6: Experiment latency graph for signals

4.2.4 Evaluation

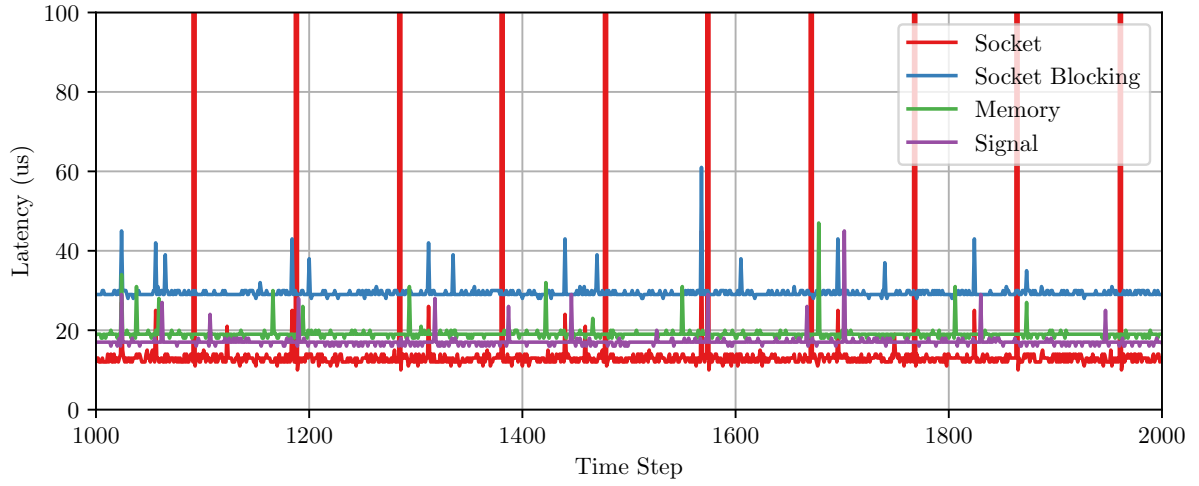


Figure 4.7: Comparison experiment latency graph for all methods

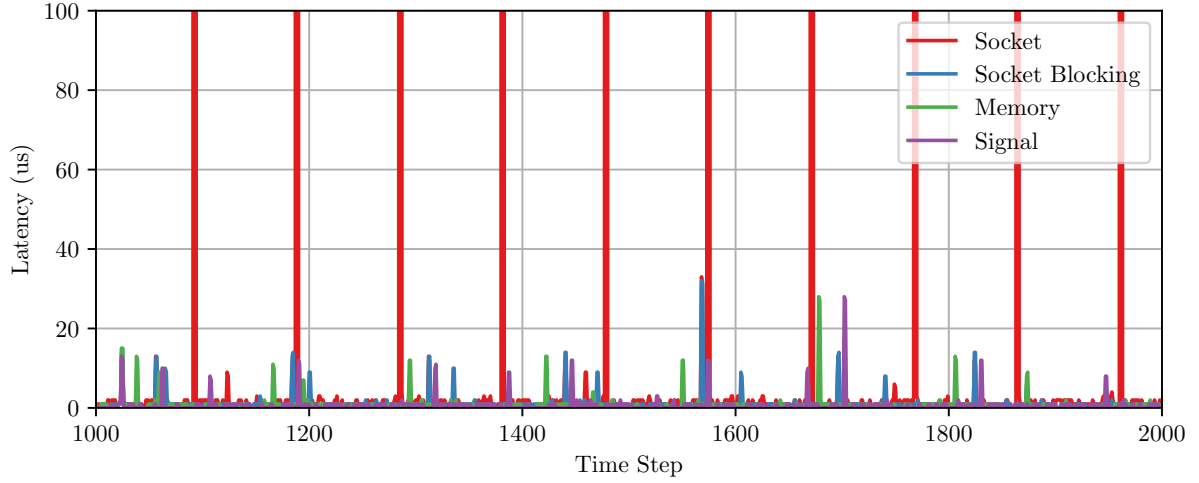


Figure 4.8: Comparison experiment jitter graph for all methods

Processor Differences

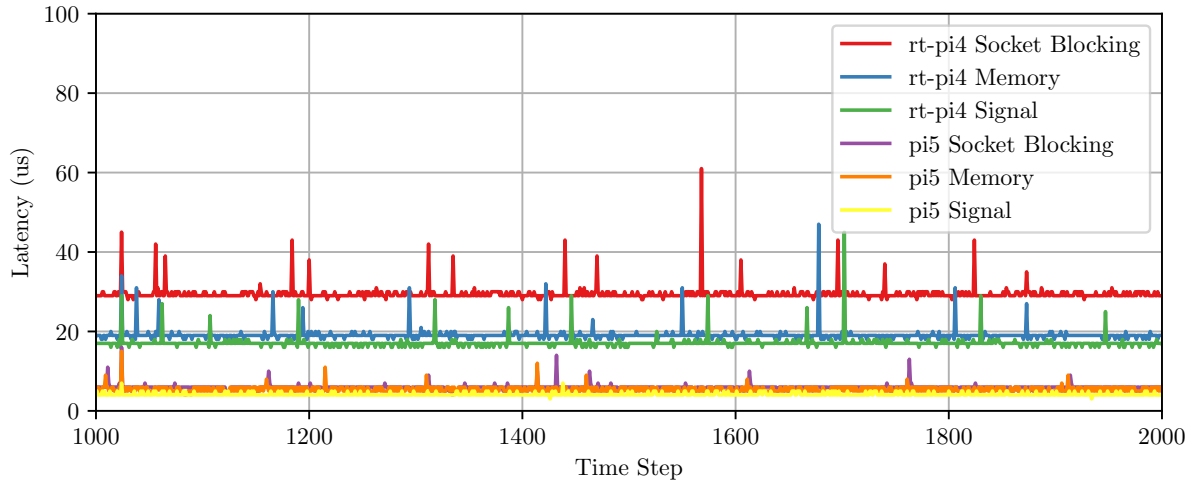


Figure 4.9: Comparison experiment latency graph for all methods on Raspberry Pi 4 vs 5

4.3 Hardware

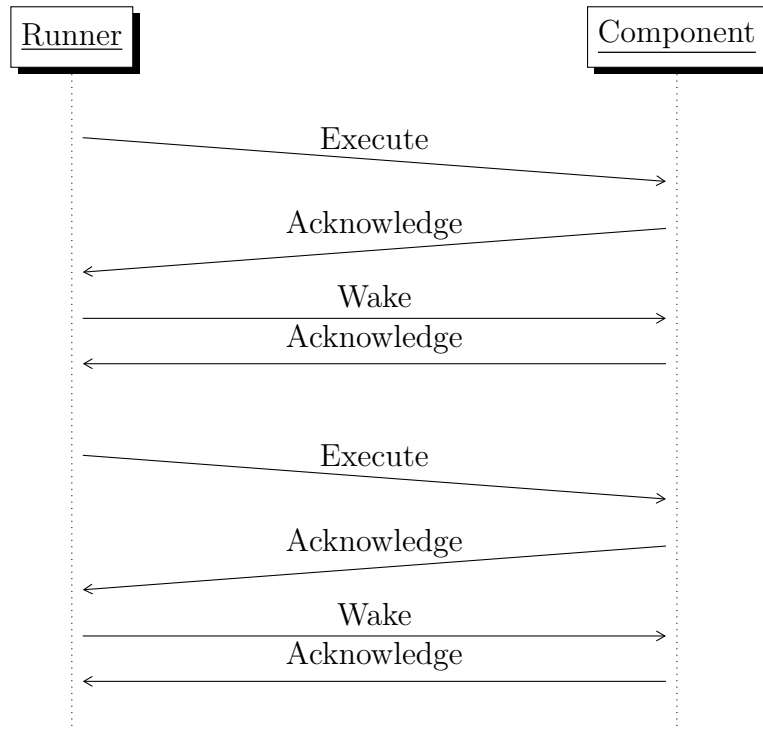


Figure 4.10: Client-Server messaging

4.3.1 Test Harness

Through this harness, an example control loop implementation can be simulated to ensure all deadlines are met during a live software update.

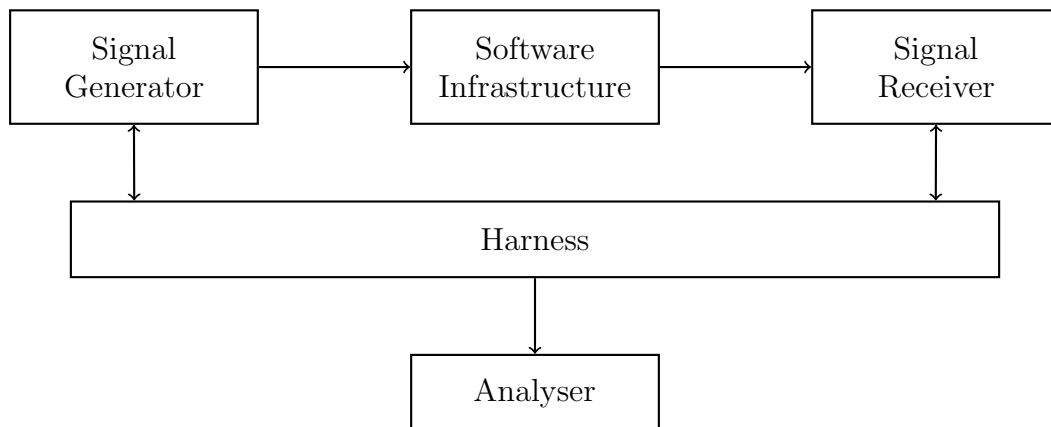


Figure 4.11: Test Harness Design

Figure 4.11 contains a signal generator, this randomly produces a signal and outputs a packet. These packets are ingested by the software infrastructure where processing occurs. An output packet is subsequently generated (mirroring the input) and sent to a signal receiver, which captures this packet.

The signal generator and receiver output their current state over discrete digital signals

so that a logic analyser can record the state of the system over time. Specifically the states can be compared to measure the complete system latency during standard operation and software update. If the generator and receiver produce additional reference clock signals, the logic analyser can more closely calculate the software infrastructure induced latency.

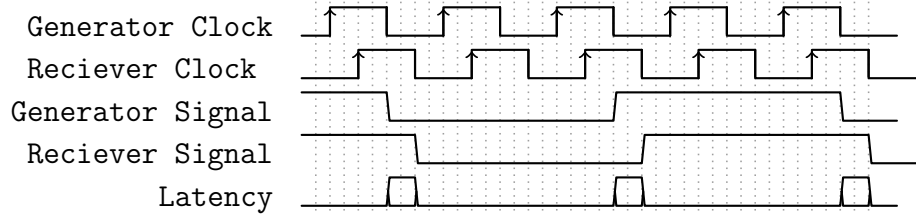


Figure 4.12: Test Harness Capture

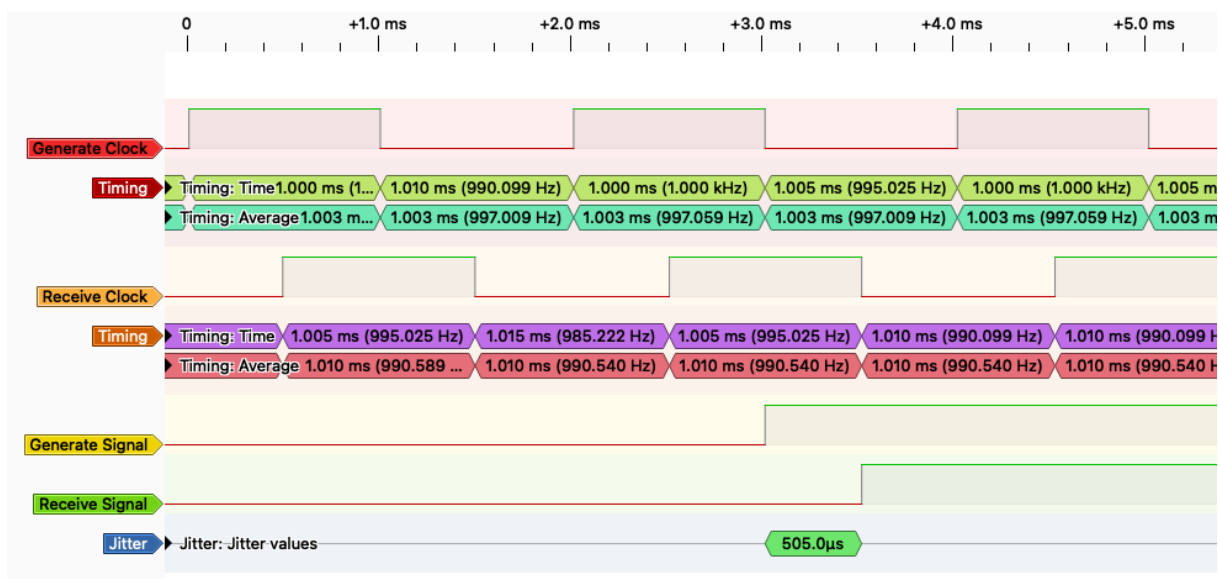


Figure 4.13: Test Harness PluseView Capture

The generator, receiver and logic analyser were all implemented using the RP2040 micro-processor from Raspberry Pi for hardware simplification.

The two controllers that performed packet communication used the W5500 chip to send UDP packets over ethernet, this integrates nicely with Elafry messages. This co-processor offloads the complex ethernet drivers and communicates to the main processor over

This communicates over SPI and offload the complex ethernet stuff onto this co-processor. SPI signals configure the IP address of the target device and allow the user to send data.

The intention for this software was to also use Rust, however we could not get the Rust libraries for the W5500 working quickly, sure with extra time with could have been possible.

To simplify this software the arduino development environment was used. This as all the drivers for the w5500 are integrated in arduino libraries.

Logic analyser <https://github.com/dotcypress/ula>

Capture the status of the 16 input channels on the Pi Pico at specified frequencies. When integrated with teh Pulseview analysis software we can idenitify the latencies

4.3.2 Development Cards

In order to build and test the real time software, dedicated development hardware is required to ensure an isolated testing environment.

These test cards consist of a Raspberry Pi Compute Module 4 (CM4). This contains the same overall hardware as the standard raspberry pi 4. EMMC flash for high vibration environments. Exception being the ethernet IC and EMMC flash (This later caused problems with compiling the operating system and bootloader)

Key requirement is remote debugging, full remote power and data control. Also placed in a rack Selected the Pi KVM project for remote control as it supports deploy as well as additional GPIO control.

The CM4s display is captured using the CSI port on the KVM with a HDMI to CSI converter. The power is controlled using a relay, that controls the 12v supply to the CM4. The signal flags of the EMMC storage are directly connected to the KVM. EXPLAIN the signal flags These are configured in the KVM user interface UART is connected to a USB serial converter. USB host is connected to the KVM for flashing the CM4. Finally a debugger (JTAG) in case of modifications to the kernel itself.

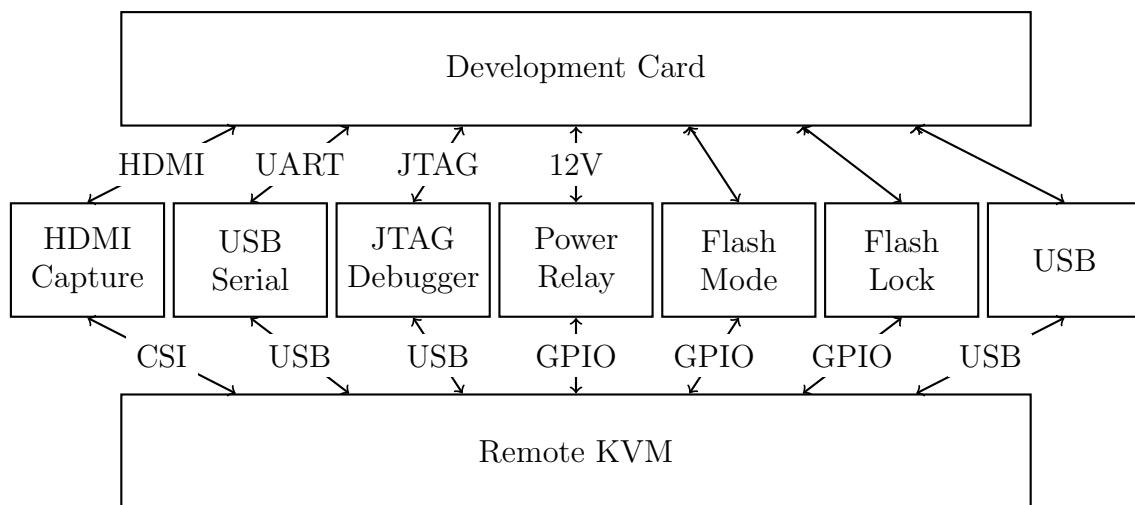
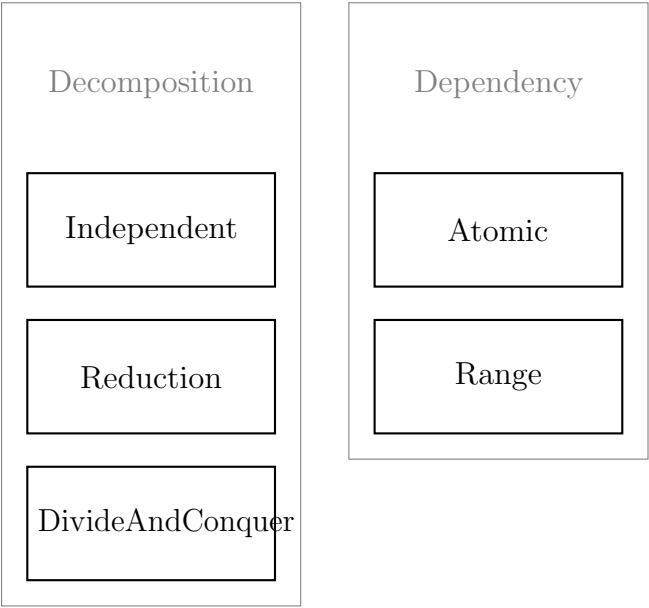
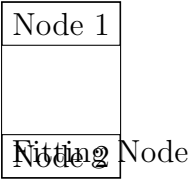


Figure 4.14: Development Hardware

Two of these so that multiple benchmarking activities can be performed at once and compared.

Use the uBoot boot loader. This allows the development images to be loaded over TFTP, we have a dedicated tftp boot server.

Connected to a managed switch. The test harness is connected to this on a separate VLAN.



Chapter 5

Implementation

```
import numpy as np

def incmatrix(genl1, genl2):
    m = len(genl1)
    n = len(genl2)
    M = None #to become the incidence matrix
    VT = np.zeros((n*m, 1), int) #dummy variable

    #compute the bitwise xor matrix
    M1 = bitxormatrix(genl1)
    M2 = np.triu(bitxormatrix(genl2), 1)

    for i in range(m-1):
        for j in range(i+1, m):
            [r, c] = np.where(M2 == M1[i, j])
            for k in range(len(r)):
                VT[(i)*n + r[k]] = 1;
                VT[(i)*n + c[k]] = 1;
                VT[(j)*n + r[k]] = 1;
                VT[(j)*n + c[k]] = 1;

            if M is None:
                M = np.copy(VT)
            else:
                M = np.concatenate((M, VT), 1)
```

```
VT = np.zeros((n*m,1), int)

return M
```

Chapter 6

Testing

Parameter	Values	Selected
kernel_size_1	2, 3, 4, 5	4
kernel_size_2	2, 3, 4, 5	3
kernel_size_3	2, 3, 4, 5	4
kernel_size_4	2, 3, 4, 5	4
pool_size_1	2, 3, 4	4
pool_siz_2	2, 3, 4	3
✓	0.2, 0.8, step=0.1	0.6
dropout_2	0.2, 0.8, step=0.1	0.5
dense_layer_units	1000, 2000, 3000, 4000, 5000	3000
learning_rate	min=0.0001, max=0.1, sampling="LOG"	0.00024

Table 6.1: Hyperparameter values for CNN

Chapter 7

Evaluation

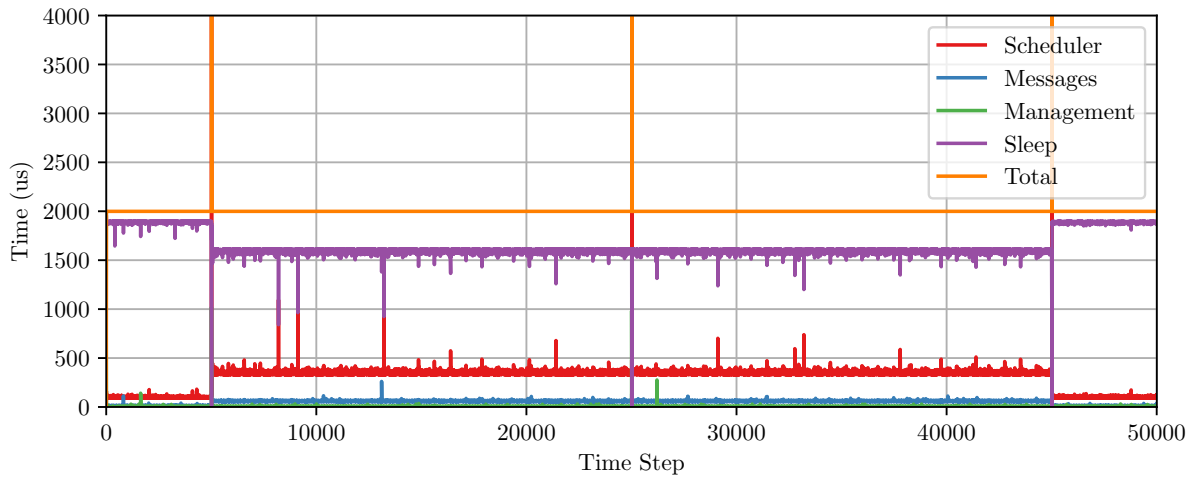


Figure 7.1: Elafry Demo Times

Chapter 8

Conclusion

References

- Abeni, Luca and Dario Faggioli (June 2020). “Using Xen and KVM as Real-Time Hypervisors”. In: *Journal of Systems Architecture* 106, p. 101709. ISSN: 13837621. DOI: 10.1016/j.sysarc.2020.101709. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1383762120300035> (visited on 10/06/2023).
- elast0ny (Apr. 3, 2024). *Elast0ny/Shared_memory*. URL: https://github.com/elast0ny/shared_memory (visited on 04/09/2024).
- Kavi, Krishna, Robert Akl, and Ali Hurson (Mar. 16, 2009). “Real-Time Systems: An Introduction and the State-of-the-Art”. In: *Wiley Encyclopedia of Computer Science and Engineering*. Ed. by Benjamin W. Wah. 1st ed. Wiley, pp. 2369–2377. ISBN: 978-0-471-38393-2 978-0-470-05011-8. DOI: 10.1002/9780470050118.ecse344. URL: <https://onlinelibrary.wiley.com/doi/10.1002/9780470050118.ecse344> (visited on 03/05/2024).
- Montgomery, Jami (July 2004). “A Model for Updating Real-Time Applications”. In: *Real-Time Systems* 27.2, pp. 169–189. ISSN: 0922-6443. DOI: 10.1023/B:TIME.0000027932.11280.3c. URL: <http://link.springer.com/10.1023/B:TIME.0000027932.11280.3c> (visited on 02/16/2024).
- Moquin, S. J. et al. (Apr. 2019). “Enhanced Uptime and Firmware Cybersecurity for Grid-Connected Power Electronics”. In: *2019 IEEE CyberPELS (CyberPELS)*. 2019 IEEE CyberPELS (CyberPELS). Knoxville, TN, USA: IEEE, pp. 1–6. ISBN: 978-1-72812-925-9. DOI: 10.1109/CyberPELS.2019.8925027. URL: <https://ieeexplore.ieee.org/document/8925027/> (visited on 10/06/2023).
- Rao, Sira et al. (Jan. 2023). “Live & Seamless Firmware Upgrade in Real Time Control Systems”. In: *2023 36th International Conference on VLSI Design and 2023 22nd International Conference on Embedded Systems (VLSID)*. 2023 36th International Conference on VLSI Design and 2023 22nd International Conference on Embedded Systems (VLSID), pp. 1–5. DOI: 10.1109/VLSID57277.2023.00021. URL: <https://ieeexplore.ieee.org/abstract/document/10089983> (visited on 10/19/2023).
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles, Global Edition*. Pearson Education. ISBN: 978-1-292-21430-6. URL: <https://books.google.co.uk/books?id=-76GEAAQBAJ>.

- Wahler, Michael and Manuel Oriol (Sept. 2014). “Disruption-Free Software Updates in Automation Systems”. In: *Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA)*. 2014 IEEE Emerging Technology and Factory Automation (ETFA). Barcelona, Spain: IEEE, pp. 1–8. ISBN: 978-1-4799-4845-1. DOI: 10.1109/ETFA.2014.7005075. URL: <http://ieeexplore.ieee.org/document/7005075/> (visited on 10/11/2023).
- Wahler, Michael, Stefan Richter, and Manuel Oriol (Oct. 25, 2009). *Dynamic Software Updates for Real-Time Systems*. DOI: 10.1145/1656437.1656440.

Appendix A

Extra Information

Some more text ...

